

SIMATS ENGINEERING

ASSESSMENT TOOL 3: Reverse Engineering Task

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA15

COURSE OUTCOME COVERED

CO4: *Analyze big data characteristics and challenges across domains including security and application aspects.* (BL4)

Dave's Taxonomy: Articulation (Level 4)
Question Count: 5

Total Marks: 30

Code of Conduct

I YADDALA KUSHAL KUMAR(192524068) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: YADDALA KUSHAL KUMAR

Assessment Tasks

Reverse Engineering Task

For each task, study the described big data solution, infer how it was built, identify the underlying components, and explain what design decisions were likely made.

1. Large-Scale Customer Interaction and Transaction Platform

The platform collects billions of website clicks, mobile interactions, and customer transactions, so it would most likely use a distributed storage architecture such as **Hadoop Distributed File System (HDFS)** or cloud object storage such as Amazon S3. HDFS provides fault tolerance by distributing data across multiple nodes and replicating blocks. Structured transaction data could be stored in relational or distributed databases such as **Apache HBase**, while semi-structured clickstream and mobile-event data could be stored in formats such as JSON, Avro, or Parquet.

For data ingestion, **Apache Kafka** would be a probable choice because it can continuously collect high-volume events from websites, mobile applications, and transaction systems. Batch processing could be performed using **Apache Spark** or Hadoop MapReduce. Spark would be especially useful because it supports large-scale transformations, SQL analytics, and machine learning.

Structured data would generally be organized into well-defined tables containing customer, transaction, product, and payment information. Semi-structured data would contain flexible event records such as clicks, page views, device information, and application logs. Data could be partitioned by date, region, customer segment, or event type. Columnar formats such as Parquet could improve analytical query performance, while indexes and distributed key-value stores could support fast lookups.

For real-time analytics, **Apache Spark Structured Streaming**, Apache Flink, or Kafka Streams could process events as they arrive. These systems could calculate metrics such as active users, transaction rates, conversion rates, and abnormal activity in near real time.

Scalability would be achieved through horizontal scaling, where additional worker nodes are added as data volume increases. Kafka topics could be divided into partitions, while storage and processing workloads could be distributed across clusters. Replication would provide fault tolerance if a server fails.

Finally, dashboards could connect to the processed analytical layer through tools such as **Power BI, Tableau, or Apache Superset**. Business intelligence systems could query an analytical warehouse or data lakehouse rather than directly querying raw event data. This architecture allows executives and analysts to monitor sales, customer behavior, traffic, and operational KPIs.

2. E-Commerce Personalization and Recommendation Platform

An e-commerce platform tracking cart additions, abandoned checkouts, product views, and payment behavior would probably use **Apache Kafka** or a similar event-streaming platform. Each customer activity could become an event containing information such as customer ID, product ID, timestamp, location, device, and event type.

A customer-profile database could use technologies such as **Redis, Apache Cassandra, MongoDB, or HBase**. Redis would be particularly useful for maintaining frequently accessed session and personalization information. Cassandra or another distributed database could store large customer profiles and activity histories.

The recommendation workflow would collect clickstream events and transaction information, process them in real time, and generate recommendations using machine-learning models. Collaborative filtering, content-based recommendation, or hybrid recommendation algorithms could be applied. A typical workflow would contain data collection, cleaning, feature engineering, model training, model evaluation, deployment, and continuous model updating.

Session data, clickstream records, and transaction records could be synchronized using Kafka event streams. A unique customer or session ID would allow events from multiple systems to be connected. Structured order information could be stored in tables, while browsing events could remain in JSON or other semi-structured formats. A data lake could store raw events, while Spark or another distributed query engine could combine them with structured order records.

Caching would reduce latency for frequently requested information such as product details, customer preferences, shopping carts, and recommendations. **Redis** could provide fast in-memory access. Distributed query engines such as **Presto/Trino or Spark SQL** could support large-scale analytical queries.

Scalability would be achieved through Kafka partitions, distributed databases, and horizontally scalable processing clusters. Replication and checkpointing would protect the system from failures. Consumer groups would allow multiple processing instances to share event-processing workloads.

Dashboards could be created using Power BI, Tableau, Grafana, or similar visualization systems. KPIs could include conversion rate, cart-abandonment rate, average order value, revenue by region, product popularity, and recommendation click-through rate. Real-time dashboards could receive processed data from a streaming analytics layer.

3. Healthcare Big Data Platform

A healthcare system processing patient histories, diagnostic images, wearable streams, and prescription records would require a **hybrid database architecture** because healthcare data contains both structured and unstructured information.

Structured information such as patient demographics, laboratory results, prescriptions, and billing records could be stored in relational databases such as PostgreSQL, MySQL, or distributed relational systems.

Medical images such as X-rays, CT scans, and MRI files could be stored in object storage or specialized medical repositories. Semi-structured records could use MongoDB or document-based storage.

Data integration between hospitals, laboratories, pharmacies, and insurance providers could use **HL7 and FHIR** standards. Integration platforms or ETL tools could extract information from different systems, transform it into standardized formats, and load it into a centralized data lake or warehouse.

Streaming frameworks such as Kafka and Spark Structured Streaming could process wearable-device data continuously. For example, heart rate, blood oxygen, temperature, or activity information could be analyzed

to identify abnormal patterns. Apache Spark or similar distributed processing frameworks could perform large-scale historical analysis.

Machine-learning systems could support disease prediction, patient-risk classification, readmission prediction, medical-image analysis, and patient monitoring. Training data could be stored in a secure data lake, processed using Spark, and supplied to machine-learning frameworks such as TensorFlow, PyTorch, or Spark MLlib.

Privacy and security would be critical. Patient information would require encryption both **at rest and in transit**. Role-based access control would ensure that doctors, nurses, researchers, administrators, and insurance personnel only access information required for their responsibilities. Audit logs would record access and modifications. De-identification or pseudonymization could be used for research datasets.

The analytics pipeline could therefore follow:

Hospital/Lab/Wearable Sources → Data Integration → Secure Data Lake → Data Cleaning → Distributed Processing → ML/Analytics → Clinical Decision Support

Clinical dashboards could display patient risk scores, abnormal sensor readings, laboratory trends, and population-health statistics while maintaining strict access controls.

4. Smart City Big Data Platform

A smart city platform continuously receiving traffic sensors, CCTV metadata, weather information, and public transportation activity would probably use **edge computing** combined with a centralized cloud or data-center platform.

Edge devices would process data close to where it is generated. Traffic cameras and sensors could perform preliminary filtering, aggregation, or event detection before transmitting information to the central system.

This reduces network bandwidth requirements and allows time-sensitive decisions to be made quickly.

A distributed messaging system such as **Apache Kafka** could collect events from traffic sensors, buses, weather stations, and other IoT devices. Kafka partitions would distribute the incoming workload across multiple brokers and provide reliable event delivery.

Centralized storage could use a data lake based on HDFS or cloud object storage. Time-series databases such as **InfluxDB or TimescaleDB** could store sensor measurements. Location-aware information could be managed using PostgreSQL with PostGIS or another geospatial database.

Streaming engines such as Apache Flink or Spark Structured Streaming could analyze traffic information in real time. Geospatial processing could identify congestion zones, calculate vehicle density, determine traffic speed, and correlate traffic conditions with weather or public transport activity.

For example:

Traffic Sensors → Edge Processing → Kafka → Stream Processing → Congestion Prediction → Traffic Control Dashboard

Historical data could be processed using Spark to identify long-term traffic patterns. Batch analytics could help city planners determine where new roads, bus routes, traffic signals, or parking facilities are needed.

Real-time processing would handle immediate operational decisions.

Security would include device authentication, encrypted communication, network segmentation, role-based access control, and continuous monitoring. Sensitive CCTV-related information should be protected through restricted access and appropriate data-retention policies.

Visualization systems such as **Grafana, Power BI, Tableau, or custom GIS dashboards** could provide live operational intelligence. City operators could monitor traffic density, public transportation status, weather conditions, incidents, and predicted congestion through a centralized dashboard.

5. Healthcare Analytics and Predictive Health Platform

This system combines patient records, wearable-device readings, laboratory reports, and appointment histories. A hybrid storage architecture would therefore be appropriate. Structured information such as patient demographics, appointments, laboratory values, and prescriptions could be stored in relational databases. Wearable data and other semi-structured information could be stored in a data lake or NoSQL database.

Medical documents and other unstructured information could be maintained in secure object storage. A lakehouse architecture could provide a common analytical platform for structured, semi-structured, and

unstructured information.

ETL or ELT pipelines would extract data from hospitals and clinics, transform it into standardized formats, and load it into the analytical environment. Healthcare interoperability standards such as **HL7 and FHIR** could allow different hospital information systems to exchange clinical information.

Real-time monitoring could use IoT gateways and Kafka to receive wearable-device measurements. Spark Structured Streaming or Apache Flink could analyze these events and generate alerts when abnormal patterns occur.

For population-level analysis, distributed processing frameworks such as **Apache Spark** would allow millions of patient records to be analyzed across multiple computing nodes. Batch processing could identify disease trends, treatment outcomes, readmission patterns, and healthcare utilization.

Machine-learning models could use patient history, laboratory results, wearable measurements, and appointment information to generate **risk scores**. Possible applications include predicting diabetes risk, cardiovascular risk, hospital readmission, or deterioration. Recommendation systems could assist clinicians by identifying potentially suitable treatments or follow-up actions, but such outputs should support rather than replace qualified clinical judgment.

Data governance would be essential. Encryption would protect patient information during storage and transmission. Role-based access control and multi-factor authentication could restrict access. Audit trails would record who accessed or modified patient information. Data masking, pseudonymization, retention policies, consent management, and compliance controls would further protect sensitive healthcare information.

The overall architecture could be represented as:

Patient Records + Wearables + Labs + Appointments → ETL/Integration → Secure Data Lake/Warehouse → Spark/Flink Processing → ML Models → Risk Scores & Analytics → Clinical Dashboards

